

Software Testing Report

ENG1 Cohort 1 Team 3

Almira Demir

Mansi Deshkar

Laina Ha

James McDermott

Harry McLean

Vadim Mitko

In our team 3's version of UniSim, we utilized 3 different types of testing:

1. *Unit testing*
2. *Integration testing*
3. *Manual testing*

Unit and integration tests are automated, while manual testing has been performed by a few potential users who tested out our game during user evaluation in the last week of semester 1.

Unit testing is necessary to check whether each logically separate blocks of code are performing as expected and whether they can be used as building blocks for later code. Unit testing is appropriate in UniSim as we have a few main Manager classes that interact with each other. To isolate errors that may appear in one of those classes, we wrote individual unit tests for their methods.

Integration testing is used by our team to check whether assets used in the game are being loaded as intended. The integration testing can save considerable time by first checking whether the assets are loaded correctly before any other logic related testing is checked.

At last, we have also used manual testing to check whether the user interface and game logic would make intuitive sense to a potential user, alongside with components of the game that could not get tested automatically. For manual testing, we have created a scenario to follow for the person testing the game and note down the differences between the expected behavior and the outcome. This is appropriate as unit and integration tests cannot possibly cover all the requirements that the game is made to offer to a user.

In our game, we have used unit testing to test whether managerial classes work as intended. We have written unit tests for the following classes: EventManager.java, BuildingManager.java, ScoreManager.java, Timer.java, AchievementManager.java, and asset tests in AssetTests.java.

Event Manager

In tests for Event Manager, we have tested whether the events function properly and are displayed in a correct way, along with the fact that all the requirements regarding events system are met

Building Manager

In `BuildingManagerTests.java`, we have written tests to check whether placing down a building works as intended, meaning the building is added to buildings array, rendered on the map, and the building count is successfully updated.

ScoreManager

When writing tests for `ScoreManager`, we made sure that the functionality of decrementing score with time is working correctly, alongside with the fact that the score will change after some combination of buildings is placed.

For example, `testUpdateScoreWithBuildingCount()` simulates how buildings are placed in the actual game and checks whether the calculated score matches the expected score, i.e. the bigger the difference in building type counts, the less is the score increase.

`testDecrementScoreWithTime()` simulates a timer going down, so in this test the score is expected to go down naturally over time.

Timer

In `TimerTests`, we have tests to check that whenever `Timer` ticks, the time is being decremented, and that `Timer` correctly updates whether it has finished ticking or not. Moreover, we also check whether `getTimeToString()` method correctly returns a formatted string that is ready to be used in our UI for the game.

AchievementManager

In `AchievementManagerTests`, we have tests to check that each achievement will be added correctly once the criteria is met.

Assets

We also test for asset availability in `AssetTests.java`, which checks which building assets are present.

Package io.github.unisim.headless

all > io.github.unisim.headless

21 tests 0 failures 0 ignored 3.622s duration

100% successful

Classes

Class	Tests	Failures	Ignored	Duration	Success rate
AchievementManagerTests	4	0	0	2.810s	100%
AssetTests	1	0	0	0.012s	100%
BuildingManagerTests	7	0	0	0.611s	100%
EventManagerTests	4	0	0	0.040s	100%
ScoreManagerTests	2	0	0	0.119s	100%
TimerTests	3	0	0	0.030s	100%

To ensure robust testing, tests are traceable between tests and requirements. The table below lists all system requirements, whether they were implemented or not, how we have tested them and where these tests are included. Most implemented features are tested automatically while some have had manual tests that can be found on the website.

Tests

ID	Implemented?	Test
FR_TIME_LIMIT	YES	TimerTests: testGetRemainingTimeToString()
FR_MAP	YES	Manual test
FR_BUILDING	YES	BuildingManagerTests: testGetBuildings()
FR_BUILDING_TYPES	YES	BuildingManagerTests: testPlaceLearningBuilding() testPlaceSleepingBuilding() testPlaceRecreationalBuilding() testPlaceEatingBuilding()
FR_SATISFACTION_BUILDINGS	YES	BuildingManagerTests
FR_OBSTACLES	YES	Manual test
FR_EVENT_TYPES	YES	EventManagerTests: testSingleEventBehaviour(), testChoiceEventBehaviour1(), testChoiceEventBehaviour2()
FR_EVENT_VARIETY	YES	EventManagerTests: testGetRandomEvent()
FR_DIFFICULTY_SELECTION	NO	

ID	Implemented?	Test
FR_DIFFICULTY_EFFECTS	No	
FR_TIMER	YES	TimerTests: testTick()
FR_BUILDING_COUNTER	YES	BuildingManagerTests:testGetBuildingCount()
FR_USER_INTERFACE	YES	Manual Test
FR_INTERACTIVE_ELEMENTS	YES	Manual Test
FR_MONEY	NO	
FR_BUYING	NO	
FR_MAP_TRANSPORT	NO	
FR_MAP_ROADS	YES	Manual Test
FR_UPGRADE_BUILDING	NO	
FR_BUILDING_TIME	NO	
FR_BUILDING_SPACE	YES	Manual Test
FR_SATISFACTION_BAR	YES	Manual Test
FR_PAUSE_GAME	YES	Manual Test
FR_UNPAUSE_GAME	YES	Manual Test
FR_COURSE_BUILDING	YES	BuildingManagerTests: testPlaceLearningBuilding()
FR_ACCOM_BUILDING	YES	BuildingManagerTests: testPlaceSleepingBuilding()
FR_REC_BUILDINGS	YES	BuildingManagerTests: testPlaceRecreationalBuilding()
FR_FOOD_BUILDING	YES	BuildingManagerTests: testPlaceEatingBuilding()